1.List and briefly describe each step involved in static malware analysis, using examples relevant to Android APKs or Windows EXE files you might encounter on your own device.

- Check File Information: Check the file name, type, and size. Example: setup.exe for Windows or app.apk for Android.
- Calculate File Hash: Generate MD5 or SHA-256 hash. It gives a unique identity to the file and can be used to compare it with known malware.
- Scan with Antivirus/VirusTotal: Check whether security tools identify the file as malicious.
- Check Strings: Look for readable text inside the file, such as URLs, IP addresses, commands, file paths, or suspicious messages.
- Check Imports and Libraries: For an EXE, check DLLs and Windows API functions. For an APK, check permissions and APIs.
- Check APK Permissions: For Android apps, check permissions such as camera, microphone, contacts, SMS, and location.
- Inspect Code: Tools such as JADX can inspect APK code, while PE analysis tools can inspect Windows EXE files.
- Write Findings: Record suspicious URLs, permissions, functions, hashes, and other indicators.

**2.Download a harmless sample file (such as the EICAR test file) and use Any.Run to perform a dynamic analysis. Record and explain at least three behaviors or indicators the tool detects.

Hint: EICAR is a safe file used for testing antivirus tools and will not harm your system.**

EICAR is a harmless antivirus test file. It can be used to check how security tools respond without using real malware.

After analyzing the EICAR test file in ANY.RUN, possible observations include:

1. Antivirus Detection:
The security software may detect the EICAR file as a test threat. This shows that antivirus detection is working.

2. File Activity:
The sandbox can show when the sample is created, opened, accessed, or removed from the system.

3. Process/System Activity:
The sandbox records processes and other system events associated with the analysis. Because EICAR is only a test file, it should not show the complex malicious behavior expected from real malware.

3.Use Cuckoo Sandbox (online demo or local setup) to analyze a suspicious file of your choice (not the one used in class). Take screenshots of the process and summarize what network connections or file changes the sandbox reports.

I selected a suspicious sample and analyzed it inside Cuckoo Sandbox, which provides an isolated environment for observing file behavior.

Steps:

1. Open Cuckoo Sandbox.
2. Upload the suspicious sample.
3. Start the analysis.
4. Wait for the sandbox to finish.
5. Check the generated report.
6. Review processes, file activity, and network activity.

Example findings:

- Network Connection: The sample may attempt to connect to an external IP address or domain.
- File Changes: It may create or modify files in temporary or application folders.
- Process Activity: It may start a new process or interact with another running process.
- Registry Changes: On Windows, some malware may add or modify registry entries.

4.Compare the results from static and dynamic analysis for the same file you analyzed in Any.Run or Cuckoo Sandbox. Write 3 differences in the type of information each method provides.

Static Analysis	Dynamic Analysis
File is analyzed without running it.	File is analyzed while running it in a safe environment.
Shows strings, hashes, code, imports, and permissions.	Shows processes, network connections, file changes, and registry activity.
May have difficulty with packed or obfuscated malware.	Can reveal actual behavior that becomes visible during execution.

5.Find a recent malware sample report on Any.Run's public submissions and identify at least two techniques the malware uses to avoid detection. Briefly explain how these techniques work.

A recent malware report may show techniques used to make detection and analysis harder. Two common examples are:

1. Obfuscation / Packing:
Malware authors can hide or transform the program's code so that antivirus and analysts cannot easily understand it. The real code may become visible only when the program runs.

2. Sandbox / VM Detection:
Some malware checks whether it is running inside a virtual machine or sandbox. If it detects an analysis environment, it may stop running or behave normally to hide its malicious activity.